ĐỀ TÀI: HỆ THỐNG MÔ PHỎNG PHÁT HIỆN VÀ XỬ LÝ DEADLOCK

Mức độ: Cơ bản (Beginner)

1. Tổng quan đề tài

Tên tiếng Anh: Deadlock Detection and Recovery Simulator

Mục tiêu: Xây dựng một phần mềm đơn giản cho phép người dùng tạo các tiến trình (Process), cấp phát tài nguyên (Resource), mô phỏng quá trình tranh chấp tài nguyên và tự động phát hiện, xử lý tình trạng bế tắc (Deadlock).

Hiểu đơn giản, hệ thống trả lời ba câu hỏi:

- Ai đang giữ tài nguyên nào?
- Có tiến trình nào đang chờ nhau dẫn đến Deadlock không?
- Nếu xảy ra Deadlock, cần làm gì để hệ thống tiếp tục hoạt động?

Đây là phần mềm mô phỏng bằng thuật toán, không trực tiếp can thiệp vào các tiến trình thật của Windows hoặc Linux.

Ví dụ thực tế

Giả sử một hệ thống có:

- P1: Chương trình in tài liệu.
- P2: Chương trình xử lý hình ảnh.
- R1: Máy in.
- R2: Máy quét.

P1 giữ máy in và yêu cầu máy quét. Trong khi đó, P2 giữ máy quét và yêu cầu máy in.

Tiến trình	Đang giữ tài nguyên	Đang yêu cầu
P1	R1 (máy in)	R2 (máy quét)
P2	R2 (máy quét)	R1 (máy in)

Nhiệm vụ của project: Phát hiện P1 và P2 đang chờ lẫn nhau, thông báo Deadlock và cho phép người dùng lựa chọn một tiến trình để kết thúc, qua đó giải phóng tài nguyên.

2. Input và Output của hệ thống

Đây là phần quan trọng nhất để xác định phạm vi project.

2.1. INPUT — Dữ liệu đầu vào

Người dùng cung cấp ba nhóm dữ liệu:

A. Danh sách tiến trình (Process): Ví dụ P1, P2, P3.

B. Danh sách tài nguyên (Resource): Ví dụ R1, R2, R3. Ở mức cơ bản, mỗi tài nguyên chỉ có một đơn vị và chỉ một tiến trình được giữ tại một thời điểm.

C. Trạng thái cấp phát và yêu cầu tài nguyên:

Tiến trình	Tài nguyên đang giữ	Tài nguyên đang yêu cầu
P1	R1	R2
P2	R2	R1

Người dùng có thể nhập các thông tin này thông qua giao diện phần mềm.

2.2. OUTPUT — Kết quả đầu ra

Với input trên, hệ thống cần xuất ra:

DEADLOCK DETECTED — Phát hiện vòng chờ giữa 2 tiến trình:
P1 → P2 → P1

Nguyên nhân: P1 đang chờ R2 do P2 giữ. P2 đang chờ R1 do P1 giữ.

Đề xuất xử lý: Kết thúc P2 và giải phóng R2. Sau đó có thể cấp R2 cho P1.

Sau khi xử lý, phần mềm hiển thị trạng thái mới: không còn vòng chờ giữa các tiến trình còn hoạt động.

Input: Process + Resource + trạng thái cấp phát/yêu cầu.

Output: Trạng thái Deadlock + đồ thị trực quan + tiến trình bị bế tắc + kết quả xử lý.

3. Quy trình End-to-End của hệ thống

Đây là toàn bộ pipeline từ khi người dùng nhập dữ liệu đến khi hệ thống xử lý xong Deadlock.

Bước	Thành phần	Hoạt động
1	INPUT	Nhập danh sách Process, Resource và trạng thái sử dụng
2	RESOURCE STATE	Lưu ai đang giữ tài nguyên, ai đang chờ
3	BUILD GRAPH	Xây dựng Wait-for Graph
4	DETECTION	Dùng DFS để kiểm tra chu trình; không có Deadlock thì hiển thị trạng thái an toàn, có Deadlock thì hiển thị vòng chờ
5	RECOVERY	Người dùng chọn tiến trình để kết thúc, giải phóng tài nguyên
6	OUTPUT	Cập nhật trạng thái, vẽ lại đồ thị và kiểm tra Deadlock lần nữa

4. Thuật toán cụ thể cho từng bước

Để giữ project đơn giản, chỉ dùng Wait-for Graph + DFS. Chưa cần Banker’s Algorithm vì đây là thuật toán tránh Deadlock, trong khi mục tiêu của đề tài là phát hiện và xử lý Deadlock.

Bước 1. Nhập và kiểm tra dữ liệu

Người dùng nhập:

```
Processes = [P1, P2]
Resources = [R1, R2]

P1 holds R1
P2 holds R2

P1 requests R2
P2 requests R1
```

Hệ thống kiểm tra rằng mỗi tài nguyên không bị hai tiến trình giữ cùng lúc và các ID hợp lệ.

Bước 2. Xây dựng Wait-for Graph

Wait-for Graph là đồ thị biểu diễn quan hệ chờ giữa các tiến trình.

- Nếu P1 đang chờ tài nguyên do P2 giữ → Vẽ P1 → P2.
- Nếu P2 đang chờ tài nguyên do P1 giữ → Vẽ P2 → P1.

Kết quả: P1 → P2 → P1 (chu trình).

Bước 3. Phát hiện Deadlock bằng DFS

DFS (Depth-First Search) là thuật toán duyệt đồ thị theo chiều sâu. Hệ thống dùng DFS để tìm chu trình:

- Bắt đầu từ P1.
- Đi theo cạnh P1 → P2.
- Từ P2 tiếp tục đi theo cạnh P2 → P1.
- Phát hiện P1 đang nằm trên đường duyệt hiện tại → Có chu trình.

Vì project giới hạn mỗi loại tài nguyên có một instance, trong mô hình này có chu trình chờ là có Deadlock.

Bước 4. Xử lý Deadlock

Ở bản cơ bản, chọn phương pháp đơn giản nhất: Process Termination (kết thúc tiến trình). Ví dụ: Chọn kết thúc P2.

- Đánh dấu P2 đã kết thúc.
- Giải phóng R2 mà P2 đang giữ.
- Xóa các yêu cầu chờ của P2.
- Cấp R2 cho P1 nếu mô phỏng có chức năng cấp phát tiếp.
- Xây dựng lại đồ thị và kiểm tra Deadlock.

Kết quả: P1 có thể tiếp tục thực hiện. Đây là thao tác trên tiến trình mô phỏng, không phải kết thúc tiến trình thật trong máy tính.

5. Công nghệ sử dụng

Đề xuất Python + Streamlit + NetworkX vì dễ triển khai, ít cần kiến thức hệ thống nâng cao và phù hợp với người mới.

Thành phần	Công nghệ	Vai trò
Ngôn ngữ	Python	Xử lý logic chính
Giao diện	Streamlit	Tạo trang nhập dữ liệu và nút bấm
Lưu trạng thái	Python dict/list	Quản lý Process và Resource
Phát hiện Deadlock	DFS tự cài đặt	Tìm chu trình trong đồ thị
Trực quan hóa	NetworkX + Matplotlib	Vẽ Wait-for Graph
Lưu dữ liệu (tùy chọn)	JSON	Lưu và nạp kịch bản

Không cần database, Docker, FastAPI hay hệ thống đa luồng thật ở phiên bản đầu tiên.

Cấu trúc thư mục đề xuất

```
deadlock-simulator/
  app.py              # Giao diện Streamlit
  simulator.py        # Qu□n lý process/resource
  graph.py            # Xây dựng Wait-for Graph
  detection.py        # DFS phát hiện Deadlock
  recovery.py         # K□t thúc ti□n trình
  visualization.py    # Vẽ đ□ thị
  examples/
    deadlock.json      # Dữ liệu ví dụ
  tests/
    test_deadlock.py
  requirements.txt
  README.md
```

6. Demo một phiên hoạt động hoàn chỉnh

Đây là ví dụ về trải nghiệm của người dùng khi chạy phần mềm:

Deadlock Simulator — Demo

1. Input: Nhập dữ liệu mô phỏng.

Process	Giữ	Yêu cầu
P1	R1	R2
P2	R2	R1

Kịch bản dữ liệu được nhập sẵn để minh họa.

2. Detection: Nhấn nút “Phát hiện Deadlock”. Hệ thống xây dựng Wait-for Graph, phát hiện chu trình P1 → P2 → P1 và hiển thị cảnh báo.

3. Recovery: Người dùng chọn P2 để kết thúc. Hệ thống giải phóng R2, xóa yêu cầu chờ của P2, xây dựng lại đồ thị và thông báo không còn Deadlock.

7. Phạm vi triển khai và tiêu chí hoàn thành

Để tránh làm project quá phức tạp, nên chốt phạm vi như sau:

Các chức năng bắt buộc:

- Nhập hoặc tạo một kịch bản gồm nhiều tiến trình và tài nguyên.
- Mô phỏng trạng thái đang giữ và đang yêu cầu tài nguyên.
- Xây dựng và hiển thị Wait-for Graph.
- Phát hiện Deadlock bằng DFS.
- Cho người dùng chọn một tiến trình để kết thúc.
- Cập nhật tài nguyên và kiểm tra lại Deadlock.

Những tính năng chưa cần làm: Đa luồng thật (thread, mutex), Banker’s Algorithm, tự động lựa chọn tiến trình tối ưu, kết nối kernel hệ điều hành, database hoặc hệ thống phân tán.

Để kiểm tra kết quả, chỉ cần xây dựng ba trường hợp thử nghiệm chính:

Trường hợp	Input	Kết quả mong đợi
Bình thường	P1 giữ R1, P2 giữ R2, không có yêu cầu chờ	Không có Deadlock
Chờ một chiều	P1 chờ R2 do P2 giữ, P2 không chờ	Không có Deadlock
Chờ vòng tròn	P1 chờ P2 và P2 chờ P1	Phát hiện Deadlock, xử lý thành công

8. Kết luận

Đề tài này có thể được hiểu theo một pipeline rất đơn giản:

Input (Processes + Resources) → Mô phỏng cấp phát → Wait-for Graph → DFS Detection → Process Termination → Output (Trạng thái mới).

Về mặt kiến thức, project giúp người mới tiếp cận năm khái niệm quan trọng của Hệ điều hành: Process, Resource, Resource Allocation, Deadlock và Recovery.

Phiên bản đầu tiên chỉ cần khoảng 2–5 tiến trình, 2–5 tài nguyên, mỗi tài nguyên một instance và một yêu cầu chờ tại mỗi thời điểm. Đây là phạm vi đủ để hoàn thành đồ án cơ bản, có giao diện trực quan, thuật toán rõ ràng và demo được quy trình xử lý Deadlock từ đầu đến cuối.